Problem 1: The "Binary Blob" Lockdown

- **The Issue:** Your pipeline downloaded the repository from GitHub as a .zip file. When n8n decompressed it, it wrapped all the files inside **one single JSON row** using randomized, unpredictable field keys (file_0, file_1). This made it impossible to run a standard text-extraction node because you couldn't hardcode variable names for a repository with unpredictable file counts.
- **The Solution:** We bypassed the rigid native UI nodes and dropped in a custom JavaScript **Code Node** running in *All Items* mode. This script programmatically grabbed every dynamic binary key, flattened the data structure, and forced every file into an independent, unified row with a standardized binary key named data.

🛑 **Problem 2: The Empty Directory Trap**

- **The Issue:** Git repositories contain structural metadata and empty directory pathing files. When your text extraction node ran, the first item it hit was an empty folder root path (0 Bytes). The pipeline kept crashing or outputting null/empty strings because the initial item had no text to extract, blocking the actual data (the README file) sitting right behind it in the queue.
- **The Solution:** We updated your JavaScript chunker code with a defensive filtering guardrail (if (!fullText || fullText.trim() === "") { continue; }). This told n8n to instantly skip past directory metadata folders and only allocate processing power to files containing real, human-readable code strings.

🛑 **Problem 3: The LLM Amnesia (Function Chopping)**

- **The Issue:** Standard text chunking cuts files strictly by line length. We realized that if we chunked a massive script at exactly 100 lines, a 110-line function would get sliced cleanly in half. Because standard LLM nodes process loops in complete isolation, the AI would have total amnesia on Chunk 2—forgetting the variables and context declared at the top of the function in Chunk 1.
- **The Solution:** You made the architectural decision to swap out the basic LLM Chain for an **AI Agent Node** mapped directly to a **Window Buffer Memory** sub-node. This creates a stateful memory bridge, allowing the AI to look back at its own recent conversation history and seamlessly stitch split functions back together in its brain.

🎯 **Where Your Pipeline Stands Right Now**

Plaintext

[Webhook] —> [HTTP Get Zip] —> [Decompress] —> [JS Normalizer] —> [Text Extractor] —>